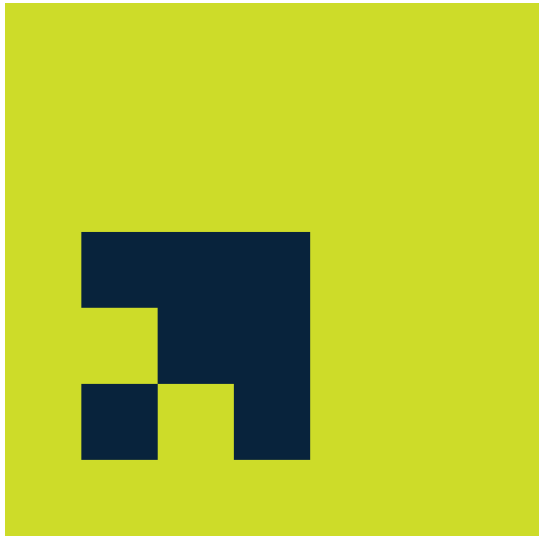
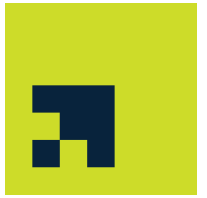


F107 – Foundations of Computer Science

Lecture 1

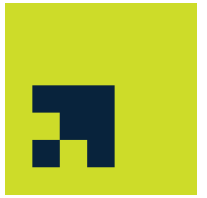
Dr. Sanjit Kumar Saha

Topics



- Foundations of Computer Science
  - Computation models
  - Computer components
  - Hardware vs. Software
  - History of computing machines
  - Types of computing machines
- Number systems
  - Decimal, binary, octal, hexadecimal systems
  - Basic arithmetic
- “Foundations of CS” by B. Forouzan
  - Chapters 1 and 2

Turing Model



- The idea of a ***universal computational device*** was first described by Alan Turing in 1936.
- Alan Turing proposed that all computation could be performed by a special kind of a machine, now called a ***Turing machine***.
- He based the model on the actions that people perform when involved in computation.
- He abstracted these actions into a model for a computational machine that has really changed the world.

Data Processor

- A computer acts as a black box
  - accepts input data
  - processes the data
  - creates output data
- For example, a ***pocket calculator*** is also a computer (which it is, in a literal sense).

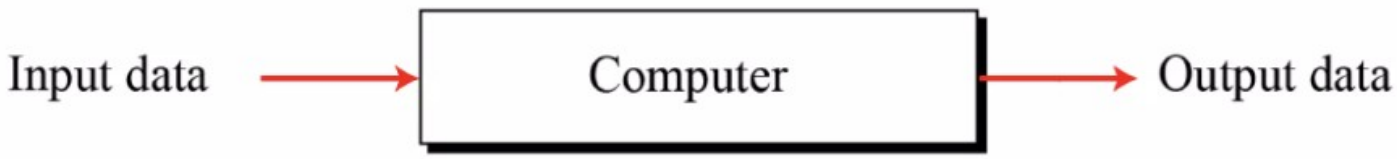


Figure 1.1 **A single purpose computing machine**

Programmable Data Processors

- The Turing model is a better model for a general-purpose computer.
- This model adds an extra element to the specific computing machine: the ***program***.
- A ***program*** is a set of instructions that tells the computer what to do with data.

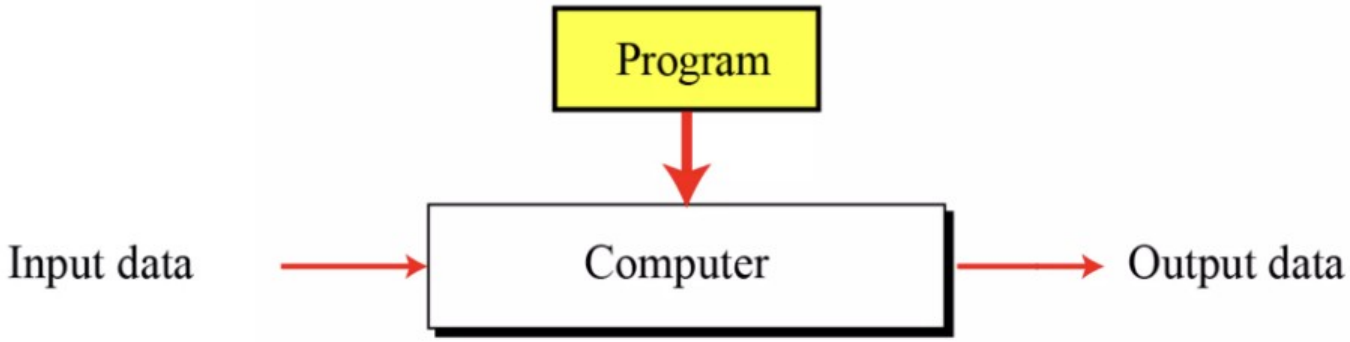


Figure 1.2 **A computer based on the Turing model**

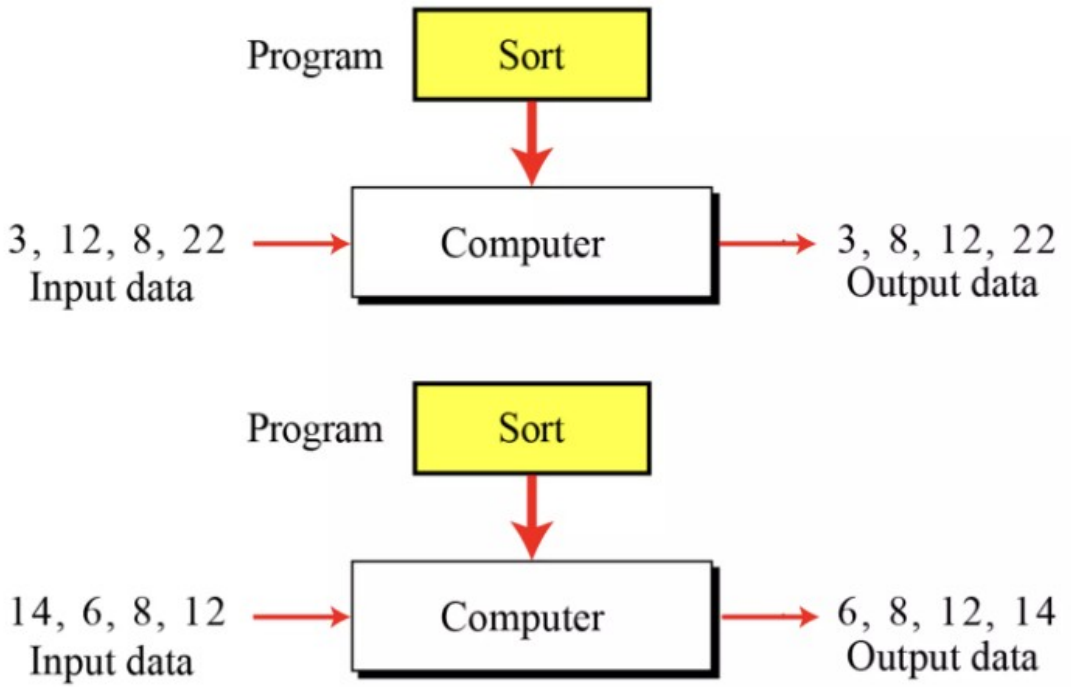


Figure 1.3 The same program, different data

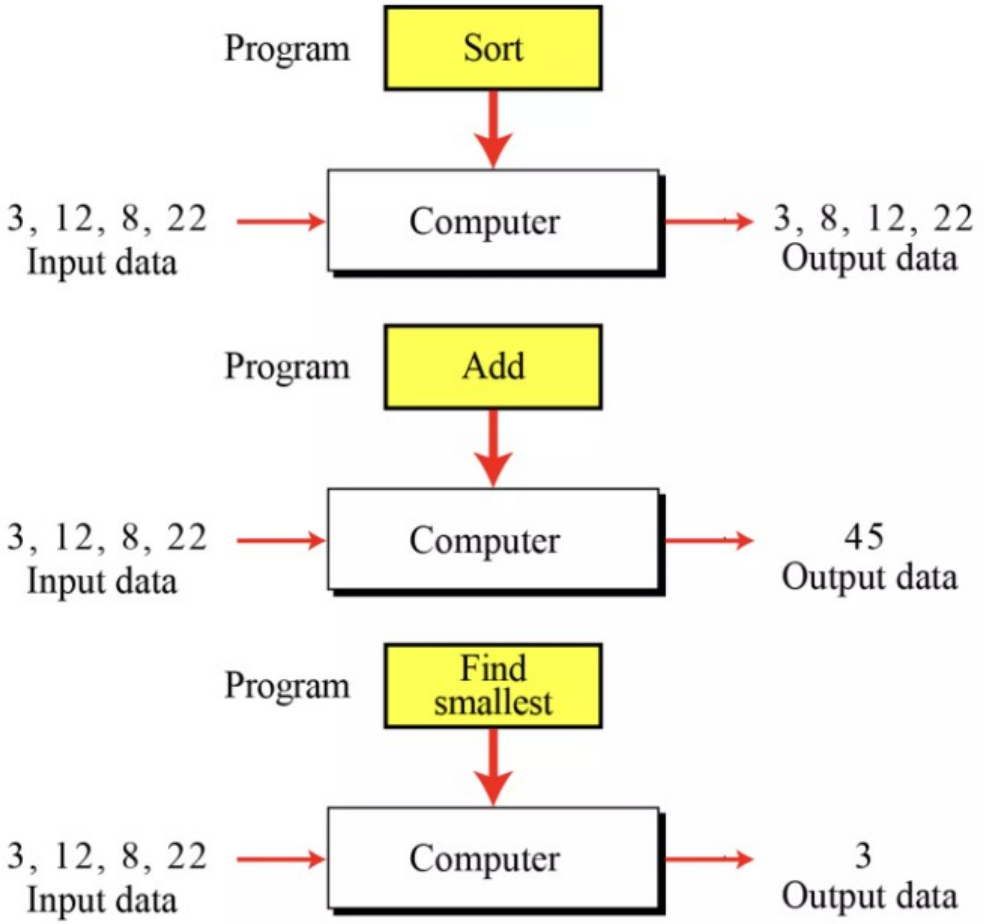
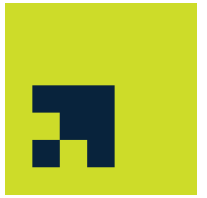


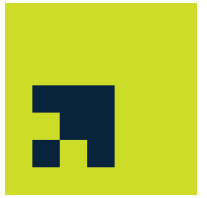
Figure 1.4 The same data, different programs

The Universal Turing Machine



- It can do any computation if the appropriate program is provided
  - the first description of a modern computer
- A very powerful computer and a universal Turing machine can compute the same thing.
  - provide the data
  - the program—the description of how to do the computation
- It is capable of computing anything that is computable.

Von Neumann Architecture Model



- Around 1944–1945, John von Neumann proposed that,
- since program and data are logically the same,
  - programs should also be stored in the memory of a computer.

Pictures: Wikipedia

Four Subsystems

- Computers built on the von Neumann model divide the computer hardware into four subsystems:
  - memory,
  - arithmetic logic unit,
  - control unit,
  - input/output

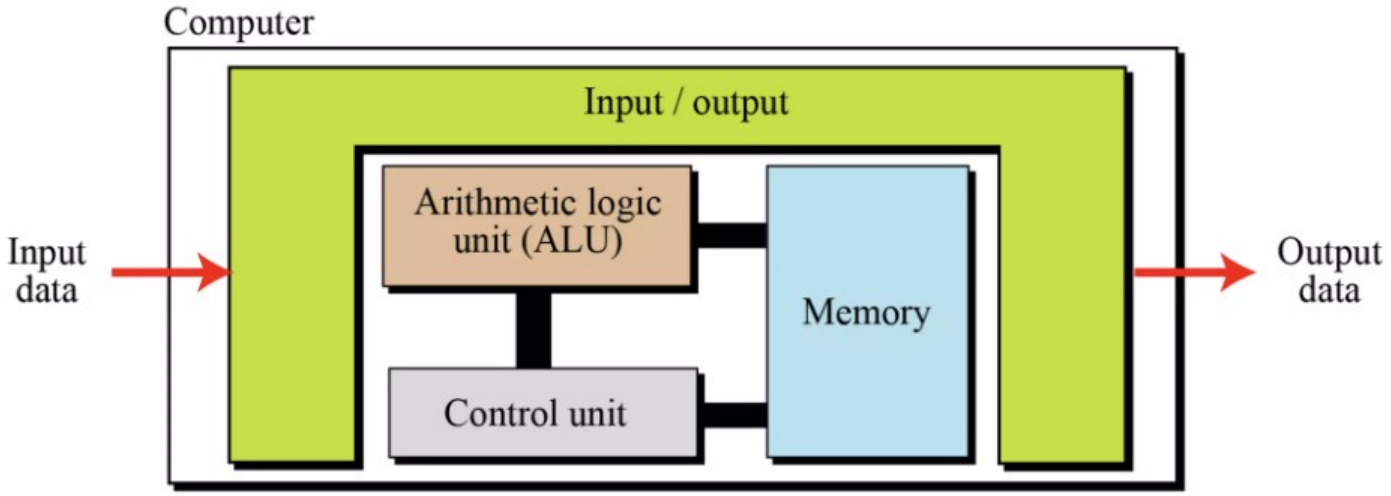
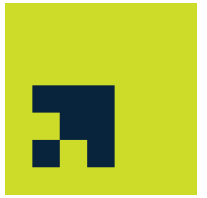


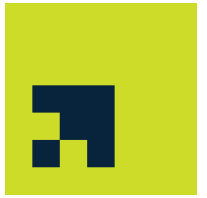
Figure 1.5 The von Neumann model

The stored program concept



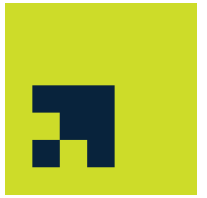
- The von Neumann model states
  - the program must be stored in memory.
- The memory of modern computers hosts both a program and its corresponding data.
  - the data and programs should have the same format, because they are stored in memory.
  - stored as binary patterns in memory—a sequence of 0s and 1s.

Components of a (Present-Day) Computer



- Hardware: physical parts
  - Mainboard
  - CPU
  - Memory (RAM, HDD, SSD, …)
- Software: operational instructions
  - Operating System
  - Programs
- Data:
  - Binary format representation of values

Programs and Algorithms

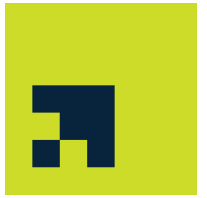


- Instruction
  - Most basic single task for a CPU
- Algorithms
  - Step-by-step methods to solve problems
- Programs
  - Sequences of instructions that realize algorithms
  - Instructions are grouped into functions
    - Allow to reuse sets of instructions to make programming scalable
  - Take inputs and generate outputs
- Programming language
  - Formal representation of instructions
  - Following specific syntax and have semantics
  - High-level and low-level languages

1. Input the first number into memory.
2. Input the second number into memory.
3. Add the two together and store the result in memory.
4. Output the result.

Program

Taxonomy of Computers



- Taxonomies based on

Computational Capabilities

- Supercomputers: extremely fast, parallel systems
- Mainframes: handle large-scale processing
- Minicomputers: devices support multiple concurrent users
- Microcomputers: devices support one user; laptops, desktops
- Embedded systems: specific-purpose devices

Purpose

- General Purpose Computers: can be tailored to needs
- Special Purpose Computers: Special setup for one task

Data Handling

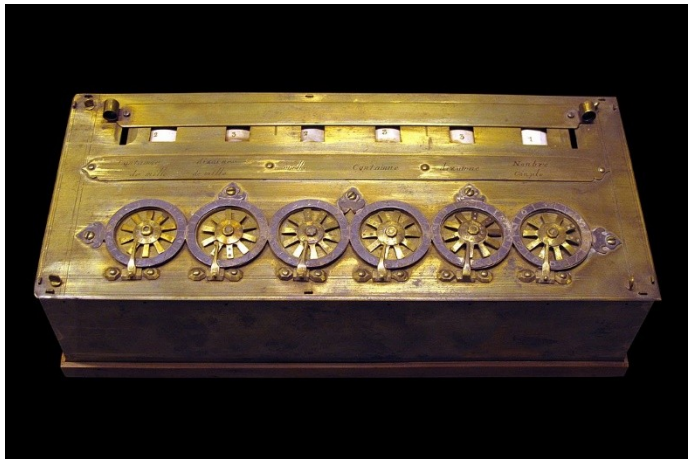
- Best-Effort System: Unpredictable performance
- Real-Time System: Guaranteed execution times

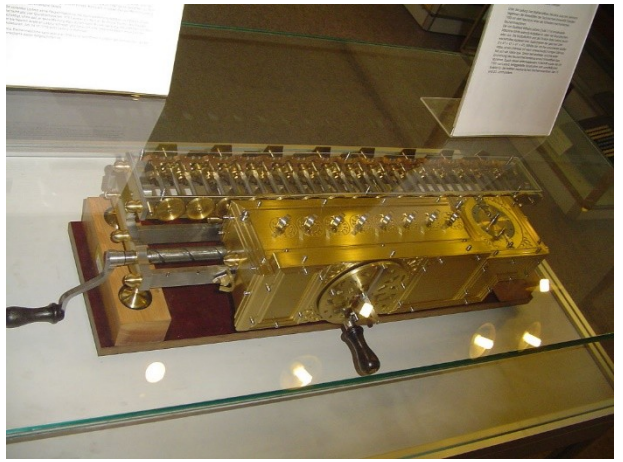
Location

- Local System: All components locally available
- Distributed System: Geographically separation of computation/storage

History of Mechanical Computing Machines

- Pascaline (1642) – mechanical adder (Pascal)
- Leibniz Wheel (17th century) – four-function calculator
- Jacquard Loom (1804) – punch card programming for looms
- Analytical Engine (1837) – Babbage created device that had ALU, memory, and control engine
- Enigma (1940) – Encryption and decryption machine for secure communication

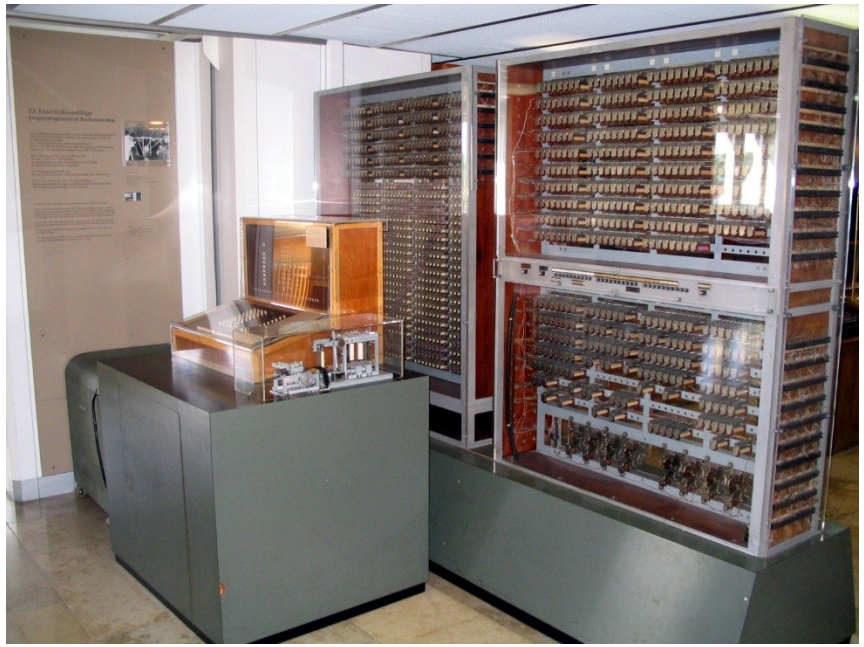


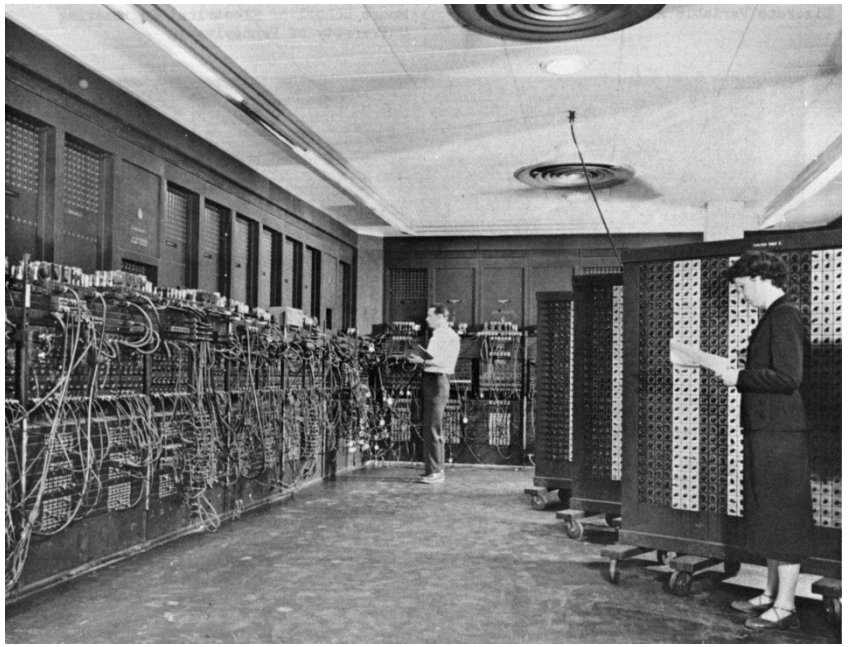


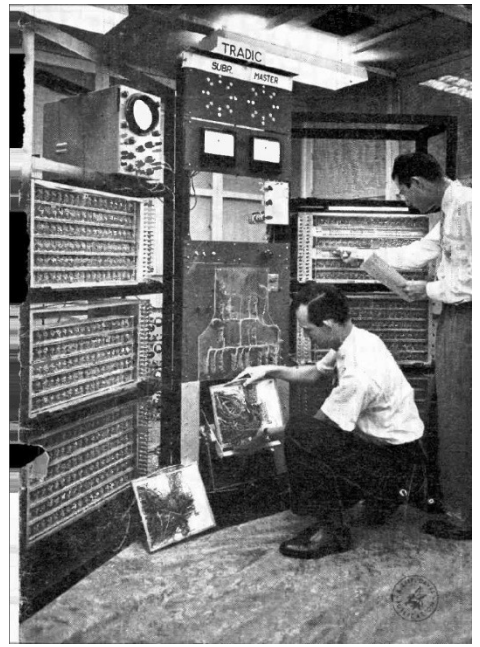
Pictures: Wikipedia

History of Electronical Computing Machines

- Z3 (1941): Konrad Zuse invented first programmable electronic computer
- ENIAC (1945): first general-purpose computer
- EDSAC (1949): first to use stored program concept
- TRADIC (1954): first transistor-based computer



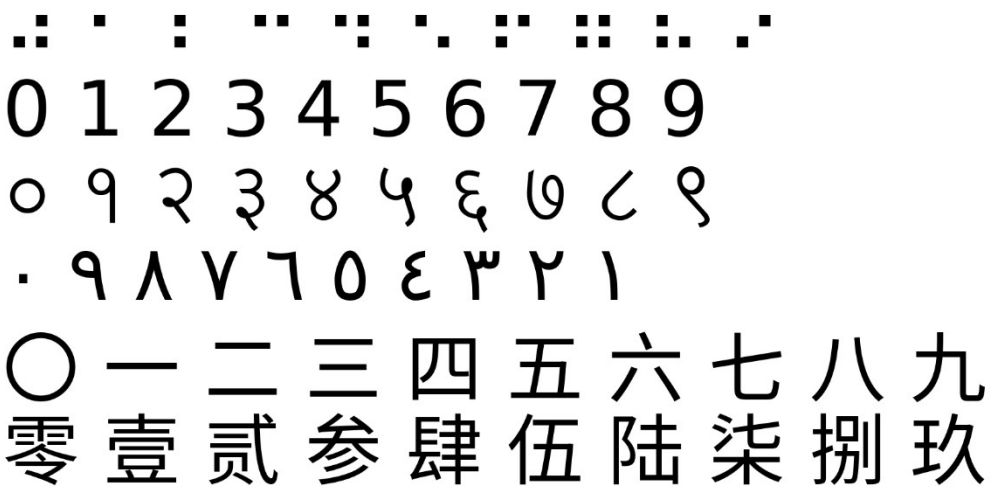




Pictures: Wikipedia

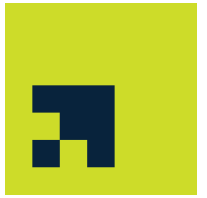
Number Systems

- A number system is a method of representing numbers using symbols
  - Positional systems
  - Non-positional systems
- It is defined by a base (radix)
- It follows either a positional or non-positional value system
- The value of the digit depends on its position



Pictures: Wikipedia

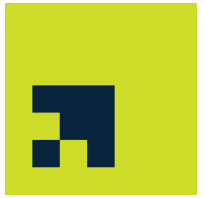
Types of Number Systems



- Decimal (Base 10)
- Binary (Base 2)
- Octal (Base 8)
- Hexadecimal (Base 16)

Pictures: Wikipedia

Decimal Number System (Base 10)



- Use digits: 0 to 9
- Most common number system used by humans
- Each position represents a power of 10
- Example: $345_{10} = (3x10^2) + (4x10^1) + (5x10^0)$

Exercises: *Rewrite the numbers in decimal number systems.*

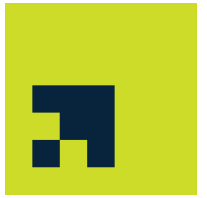
$9290_{10} =$

$56023_{10} =$

$2566221_{10} =$

Pictures: Wikipedia

Binary Number System (Base 2)



- Use digits: 0 and 1
- Used by computers
- Each digit is called a *bit*
- Each position represents a power of 2
- Example: $10110_2 = (1x2^4) + (0x2^3) + (1x2^2) + (1x2^1) + (0x2^0)$

Exercises: *Rewrite the numbers in binary number systems.*

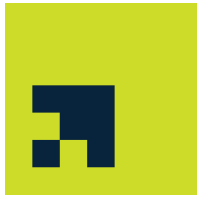
$0_2 =$

$111111_2 =$

$101011110000_2 =$

Pictures: Wikipedia

Octal Number System (Base 8)



- Use digits: 0 to 7
- Compact form of binary numbers
- Each octal digit represents 3 binary bits
- Each position represents a power of 8
- Example: $1256_8 = (1x8^3) + (2x8^2) + (5x8^1) + (6x8^0)$

Exercises: *Rewrite the numbers in octal number systems.*

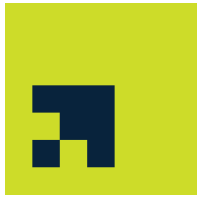
$7412_8 =$

$59673_8 =$

$6352742_8 =$

Pictures: Wikipedia

Hexadecimal Number System (Base 8)



- Use digits: 0 to 9 and A-F
- Each hex digit represents 4 binary bits
- Commonly used in programming and memory addresses
- Each position represents a power of 16
- Example: $2AE_{16} = (2x16^2) + (10x16^1) + (14x16^0)$

Exercises: *Rewrite the numbers in hexadecimal number systems.*

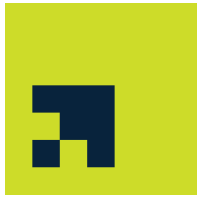
$74648_{16} =$

$59F73_{16} =$

$6AC52E4D_{16} =$

Pictures: Wikipedia

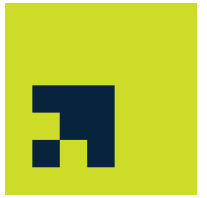
Conversions to Base-10



- Use the previous representation and add up the values

- Exercise: Transform the following numbers to Base-10 representation
  - $1101_2 =$
  - $043_8 =$
  - $3210_8 =$
  - $1234_{16} =$
  - $100\mathrm{F}_{16} =$

Conversions from Base-10



- Division with remainder (rest)
  1. Divide number given in Base-10 by new base
  2. Write down remainder of the division, even if 0 (from right to left)
  3. If the whole part of the division is not zero, go back to Step 1; if it is zero, stop

- Example: $561_{10} = ?_2 = ?_{16}$

Real Numbers vs. Integer Numbers

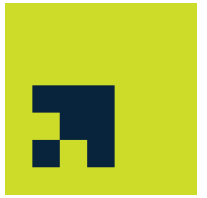
- Integers $\mathbb{Z}$: No fractional part
- Reals $\mathbb{R}$: Include fractional part

- How to represent fractions? → Different format required
  - Precision
  - Storage
  - Computational effort

- Reals are represented as “floats” (IEEE 754)
  - Floating-Point and Fixed-Point Notations
  - Example: 1001.011_2

See Session 3

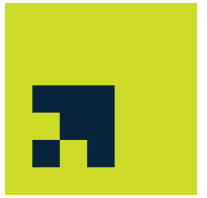
Arithmetic in Base-2



- Arithmetic works in any base the same
  - Overflows occur, but must respect the new base

- Addition: $90_{10} + 54_{10} = ?_{10}$

Arithmetic in Base-2



- Multiplication: $56_{10} \times 84_{10} = ?_{10}$

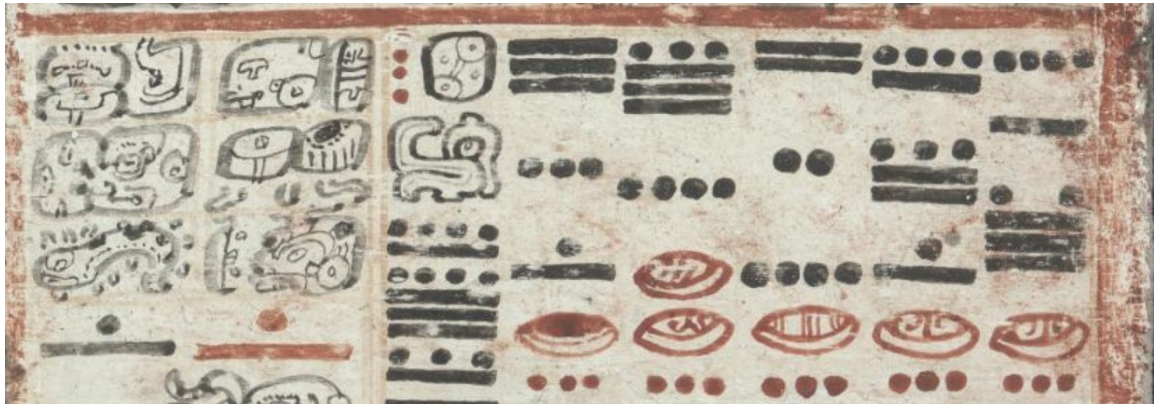
Non-Positional Number Systems

- Symbols have fixed values regardless of position
  - Some inherent conventions to represent numbers
  - Less compact and less scalable
  - Used historically before positional systems

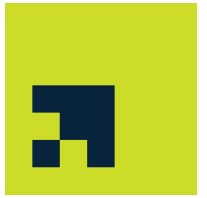
- Example: Roman numerals
  - $\mathrm{I} = 1, \mathrm{III} = 3, \mathrm{IV} = 4, \mathrm{XLIV} = 44$
  - $\mathrm{MMXXV} =$

I	V	X	L	C	D	M
1	5	10	50	100	500	1000

- Extra: Example: Mayan system



Why Number Systems are Important



- Help computers store and process data
- Enable efficient representation of information
- Used in programming, memory, networking, and hardware design

Task 1

- Explain the key differences between the Turing and Von Neumann models

Task 2

- Create a taxonomy diagram showing types of computers with examples

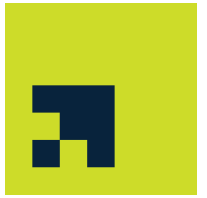
Task 3

- Describe the historical significance of the Jacquard Loom

Task 4

- List and describe the four components of the Von Neumann architecture

Task 5



- What makes a system 'Turing complete'?

Task 6

- Transform the following numbers to Base-10 representation
  - $1101110_2 =$
  - $1010110011110000_2 =$
  - $01243_8 =$
  - $7772222_8 =$
  - $97141\text{AFFE}1234_{16} =$
  - $\text{DEADCAFE}_{16} =$

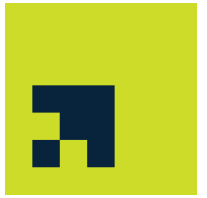
Task 7

- Transform the following numbers to Base-2, Base-8, and Base-16:
  - $590_{10} =$
  - $9999_{10} =$
  - $1000000_{10} =$

Task 8

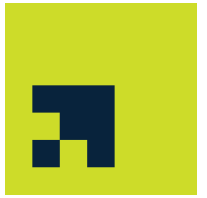
- Add and multiply the following two numbers in Base-10 and in Base-2
  - $567_{10} + 845_{10} =$
  - $567_{10} \times 845_{10} =$

Task 9



- Describe the steps to convert a fractional Base-2 number to Base-10

Task 10



- Describe floating-point representation and mention its limitations